

Práctica 3 – Broker de mensajes

En las arquitecturas de software distribuidas, como los sistemas basados en microservicios, la comunicación síncrona (por ejemplo, peticiones HTTP REST) puede generar un fuerte acoplamiento entre componentes y problemas de rendimiento si un servicio se bloquea esperando la respuesta de otro.

Para solucionar esto, se emplea el patrón arquitectónico **Broker de Mensajes** (o *Message-Oriented Middleware*, MOM), ilustrado en la Figura 1. Un broker actúa como un intermediario que permite la comunicación asíncrona entre aplicaciones. Su funcionamiento básico se basa en desacoplar a los creadores de la información de los consumidores de la misma. Existen por tanto las siguientes entidades:

- **Productores (*Publishers*)**: Son los componentes que generan datos o eventos y los envían al broker. No necesitan saber quién va a recibir el mensaje.
- **Consumidores (*Subscribers*)**: Son los componentes que se conectan al broker para leer y procesar los mensajes. No necesitan saber quién los generó.
- **Colas (*Queues*)**: Estructuras de almacenamiento dentro del broker donde los mensajes se guardan de forma segura hasta que los consumidores están listos para procesarlos.

El uso de un MOM garantiza la entrega de mensajes, balancea la carga de trabajo y

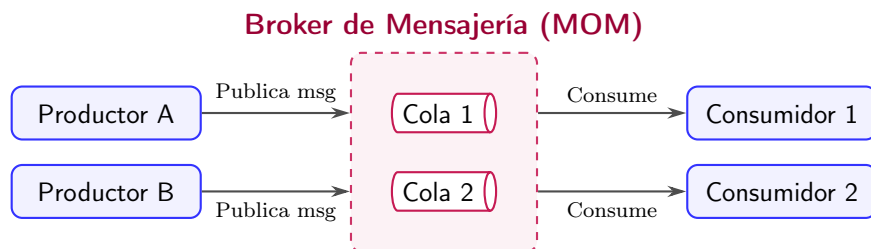


Figura 1: Patrón arquitectónico de un Broker de Mensajes.

permite que el sistema escale de forma independiente. **IMPORTANTE:** Esta práctica no tendrá una calificación numérica independiente, sino que se evaluará y defenderá de forma conjunta con la siguiente entrega de la asignatura. Esto se debe a que el *broker* de mensajería que se desarrolle aquí será una pieza clave y obligatoria para implementar la orquestación asíncrona en la próxima práctica.

1. Desarrollo del Broker de Mensajes

Para comprender a fondo cómo funciona un broker en la industria, se recomienda realizar los tutoriales de RabbitMQ descritos en el **Anexo A** antes de comenzar el desarrollo.

El objetivo de esta práctica principal es **desarrollar nuestro propio broker de mensajes**. Esta práctica se plantea con diferentes niveles de dificultad, permitiendo que cada equipo adapte el alcance de su desarrollo según su ritmo e intereses. Los niveles están pensados para evolucionar desde una arquitectura básica hasta una solución más completa y robusta.

La solución debe desarrollarse en *Python* o *Java*, pero sin utilizar RabbitMQ ni ninguna otra librería de MOM preexistente.

Versión básica

Las funcionalidades del MOM serán similares a las de los tutoriales 1 y 2 de RabbitMQ. Permitirá que un programa *productor* deposite mensajes en una cola y que un programa *consumidor* los lea. Deberá ofrecer, al menos, los siguientes servicios:

Servicio 1.- `declarar_cola(parámetros)`: operación idempotente; si se invoca varias veces con el mismo nombre, sólo se crea una única cola. Productores y consumidores la utilizarán. Aquí el patrón *Singleton* [1] puede ser de utilidad.

Servicio 2.- `publicar(parámetros)`: operación que invoca el productor para publicar mensajes en una cola previamente declarada.

Servicio 3.- `consumir(parámetros)`: operación invocada por el consumidor para recibir mensajes de una cola ya declarada.

Consideraciones adicionales:

- Si se publica en una cola inexistente, el mensaje se descarta.
- El consumidor implementará un *callback* que el MOM invocará para procesar cada mensaje.

- Si no hay consumidores disponibles, el mensaje permanecerá en la cola durante un máximo de 5 minutos; pasado este tiempo, el MOM lo eliminará.
- Si varios consumidores están suscritos a la misma cola, los mensajes se distribuirán en *round robin*, como se expresa en el tutorial 2 de RabbitMQ.

Versiones avanzadas

Las funcionalidades siguientes presentan versiones avanzadas del MOM desarrollado. Se plantean las siguientes:

- Listado y eliminación de colas existentes.
- Reconocimiento (ACK) de mensajes procesados (tutorial 2).
- Implementación de política *fair dispatch* (tutorial 2).
- *Message durability*: recuperación de colas y mensajes duraderos tras una caída (tutorial 2).
- Ejecución de productores y consumidores en máquinas diferentes.

2. Entrega final

1. **Código fuente documentado**, incluyendo comentarios adecuados que faciliten la comprensión del sistema implementado.
2. **Documentación de la arquitectura**, que debe incluir:
 - Vista de **módulos**.
 - Vista de **componentes y conectores**.
 - Vista de **distribución**.

Extensión máxima: 2 páginas (una hoja a doble cara).

A. RabbitMQ

RabbitMQ es uno de los brokers de mensajes más utilizados en la industria. Proporciona comunicación asíncrona entre aplicaciones mediante colas y es capaz de soportar distintos modelos de interacción; en esta sección lo utilizaremos para ilustrar el modelo de Publicación/Suscripción (P/S) estudiado en clase.

Nos familiarizaremos con RabbitMQ siguiendo sus tutoriales oficiales (accesibles en www.rabbitmq.com/getstarted.html). Realizaremos los cinco primeros tutoriales, disponibles en varios lenguajes de programación (Python, Java, Ruby, PHP, entre otros). Elige el que prefieras.

En primer lugar, descarga e instala RabbitMQ para el sistema operativo que vayas a utilizar. Basta con visitar la sección *Installation Guides* en <https://www.rabbitmq.com/download.html>.

Tras la instalación, deja el servidor en ejecución (lanzando el **broker**) para que escuche eventos, tal como indica la guía de instalación. A continuación, completa los siguientes cinco tutoriales en orden.

A.1. Tutoriales de RabbitMQ

1. Hello World! (Java | Python)

Este tutorial muestra cómo una aplicación envía mensajes a una cola y otra los consume: el mecanismo básico de comunicación asíncrona.

2. Work queues (Java | Python)

Aprenderás a enviar mensajes a una cola para que los procesen varias aplicaciones. Útil para tareas computacionalmente pesadas: el **broker** distribuye los mensajes entre los consumidores con una política *round robin*. Más adelante verás *fair dispatch* para equilibrar la carga y el uso de ACKs, que garantiza el procesamiento incluso si el broker falla.

3. Publish/Subscribe (Java | Python)

Pasamos ya a P/S. La cola deja de ser compartida: el publicador ni siquiera sabe de la existencia de las colas; son los suscriptores quienes las crean y las utilizan en exclusiva. El componente que reparte los mensajes se denomina *exchange*. El componente de este ejemplo es de tipo **fan-out**, es decir, reenvía cada mensaje a todas las colas que estén vinculadas (*binding*) a él.

4. Routing (Java | Python)

Aquí aprenderás a recibir únicamente los mensajes que te interesen. Para ello, se utiliza una *key* asociada a cada mensaje, similar al concepto de *topic*. En el ejemplo se emplea **severity**. Cada suscripción se crea mediante un *binding* entre la cola,

un *exchange* y un valor concreto de la *key* (por ejemplo, **warning**). El *exchange* debe ser de tipo *direct*.

5. **Topics** (Java | Python)

Mientras que el tipo *direct* permite filtrar por una única *key*, el *exchange* de tipo *topic* permite definir claves jerárquicas separadas por puntos, como por ejemplo `source.severity`. Así podrás publicar y suscribirte usando patrones como `kernel.warning`: avisos (**warning**) procedentes del **kernel**.

Referencias

- [1] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1st edition, October 1994.